KOENIG

step forward



OOP Deep Dive

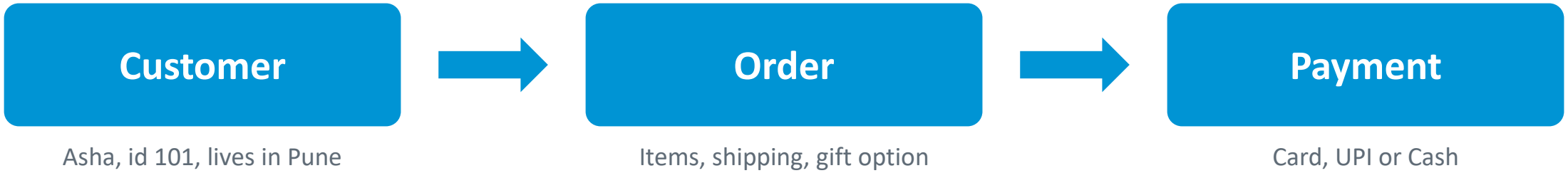
Core Java (Advanced)

OOP Deep Dive

1. The four pillars of OOP
2. Abstract classes vs interfaces
3. Composition over inheritance
4. equals(), hashCode(), toString()
5. Immutability and defensive copying
6. Records, sealed classes, pattern matching

Demo Scenario: ShopKart

ShopKart is a small online shop. We start with one product and grow it, topic by topic, until a customer can place and pay(simulation) for an order.

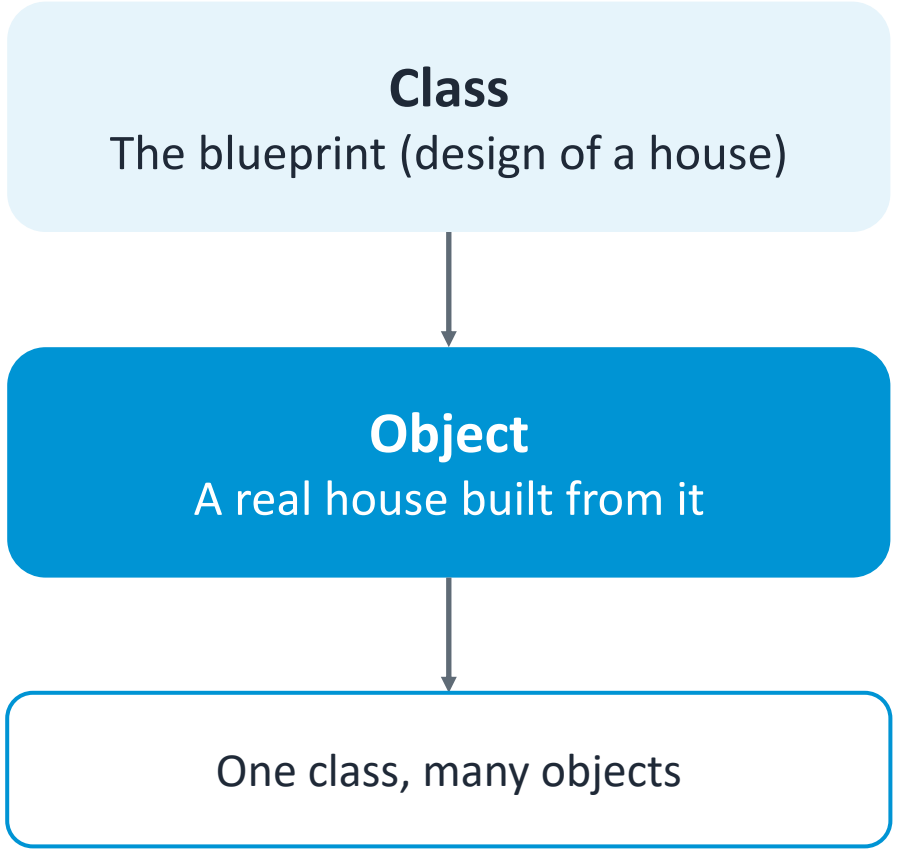


Supporting cast

- Product
- Item
- Money
- Address
- Customer
- Category

Why Object-Oriented Programming?

- **Real-world thinking:** model things like Customer, Order, Product as objects.
- **Organised code:** data and the actions on that data live together.
- **Reuse:** write once, use in many places.
- **Easy to change:** fix one class without breaking the rest.



Classes and Objects in Java

- **Fields** store the state (data).
- **Methods** define the behaviour (actions).
- **new** creates an object from the class.
- **Each object** has its own copy of the fields.

```
public class Car {
    String color;          // field (state)
    int speed;             // field (state)

    void accelerate() {    // method (behaviour)
        speed += 10;
    }
}

Car myCar = new Car();     // create an object
myCar.color = "Red";
myCar.accelerate();        // speed is now 10
```

The Four Pillars of OOP

1 **Encapsulation**

Hide the data, expose safe methods.

2 **Inheritance**

A child class reuses a parent class.

3 **Polymorphism**

One call, many behaviours.

4 **Abstraction**

Show what it does, hide how.

We revisit each pillar with code in the next slides

Encapsulation: Protect Your Data

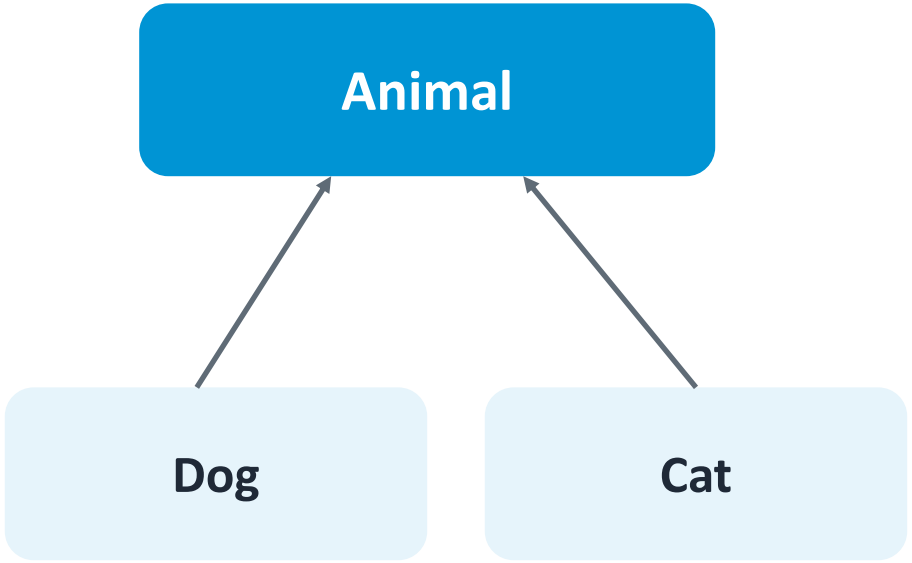
- **Make fields** private so nobody can change them directly.
- **Expose** public methods that check the rules.
- **Benefit:** invalid data can never sneak in.
- **Think:** an ATM - you press buttons, you never touch the cash vault.

```
public class BankAccount {
    private double balance;   // hidden

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException(
                "Amount must be positive");
        }
        balance += amount;
    }

    public double getBalance() {
        return balance;
    }
}
```

Inheritance: Reuse Through 'is-a'



- **A Dog is-an Animal:** it gets eat() for free.
- **extends** creates the parent-child link.
- **Good for:** clear 'is-a' relationships with shared behaviour.

```
class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Woof!");
    }
}

Dog d = new Dog();
d.eat();    // inherited from Animal
d.bark();   // Dog's own method
```

Polymorphism: Many Forms

- **Same method call,** different result depending on the real object.
- **Parent type** can hold any child object.
- **Benefit:** add new shapes without changing the loop.

```
abstract class Shape {
    abstract double area();
}
class Circle extends Shape {
    double r;
    Circle(double r) { this.r = r; }
    double area() { return Math.PI * r * r; }
}
class Rectangle extends Shape {
    double w, h;
    Rectangle(double w, double h) {
        this.w = w; this.h = h;
    }
    double area() { return w * h; }
}

List<Shape> shapes =
    List.of(new Circle(2), new Rectangle(3, 4));
for (Shape s : shapes) {
    System.out.println(s.area()); // each its own way
}
```

Abstraction: What, Not How



- **Abstraction** hides complex details behind a simple contract.
- **Driving a car:** you use the pedals, not the engine internals.
- **In Java:** achieved with abstract classes and interfaces.

```
interface PaymentMethod {
    void pay(double amount);
}
```

Demo: Four Pillars in ShopKart

What we build

- **Product:** starts as one class, then gets private stock and safe methods.
- **Product types:** PhysicalProduct and DigitalProduct extend it.
- **Payments:** Card and UPI sit behind one PaymentMethod contract.

How the concept helps

- **Encapsulation** bad stock values can never get in.
- **Inheritance** new product types reuse the shared code.
- **Polymorphism** one loop handles every product type.
- **Abstraction** checkout does not care how payment works.

Abstract Class vs Interface

Abstract Class

Use for 'is-a' with shared code.

- Can hold fields (state)
- Can have constructors
- Mix of finished and abstract methods
- A class extends only ONE

Interface

Use for 'can-do' abilities.

- A pure contract of behaviour
- No instance fields
- Mix of finished and abstract methods
- A class can implement MANY

Rule of thumb: prefer interfaces; use abstract classes to share code

Modern Interface Methods

- **abstract:** must be implemented.
- **default:** ready-made, can be overridden.
- **static:** utility on the interface itself.
- **private:** helper hidden inside the interface.

```
public interface Greeter {
    String name();                  // abstract

    default String greet() {        // default
        return decorate("Hello, " + name());
    }

    static Greeter of(String n) { // static
        return () -> n;
    }

    private String decorate(String s) { // private
        return "*** " + s + " ***";
    }
}
```

Demo: Abstract Class vs Interface

What we build

- **Product:** an abstract class holding the shared name and price.
- **Abilities:** Shippable and Taxable added only where they make sense.
- **Notifier:** an interface with default, static and private methods.

How the concept helps

- **Abstract class** shares state and code for an 'is-a' family.
- **Interface** adds 'can-do' abilities, and a class can have many.
- **An e-book** is taxable but never shippable, so it skips Shippable.

Composition Over Inheritance

Inheritance (is-a)

Deep chains become rigid and hard to test.

Composition (has-a)

Build objects from small, swappable parts.

```
class Engine {
    void start() { /* ... */ }
}

class Car {
    private final Engine engine; // has-a

    Car(Engine engine) {
        this.engine = engine;
    }
    void start() {
        engine.start();          // delegate
    }
}
```

- Car HAS-A Engine - it is not an Engine.

Program to Interfaces

- **Declare variables** using the interface type.
- **Swap implementations** later without changing callers.
- **Easier testing:** plug in a fake version.

```java
// Good: depends on the interface
List<String> names = new ArrayList<>();

// Limiting: tied to one implementation
ArrayList<String> names2 = new ArrayList<>();

interface PaymentService {
    void pay(double amount);
}

class Checkout {
    private final PaymentService payments;
    Checkout(PaymentService payments) {
        this.payments = payments;
    }
}
```

Demo: Composition Over Inheritance

What we build

- **The problem:** Express, Gift and ExpressGift order classes keep multiplying.
- **The fix:** one Order built from a ShippingPolicy and a GiftOption.
- **Checkout:** takes any PaymentService, real or fake.

How the concept helps

- **Has-a parts** can be mixed freely with no new subclass.
- **Interfaces** let us swap implementations without touching callers.
- **A fake payment** lets us test checkout without calling a bank.

equals(), hashCode(), toString()

- **equals():** when are two objects the same?
- **hashCode():** number used by HashMap and HashSet.
- **The contract:** equal objects MUST have equal hash codes.
- **toString():** readable text for logs.

```java
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Customer other)) return false;
    return id == other.id;
}

@Override
public int hashCode() {
    return Long.hashCode(id);
}

@Override
public String toString() {
    return "Customer{id=" + id + "}";
}
```

Always override equals() and hashCode() together

Demo: equals(), hashCode(), toString()

What we build

- **Customer:** the same id loaded twice, as on two different screens.
- **HashSet and HashMap:** count customers and look up a loyalty tier.
- **Fix in steps:** equals first, then hashCode.

How the concept helps

- **Without them** duplicates appear and the GOLD tier is lost.
- **equals alone** is not enough for hash-based collections.
- **toString** turns log lines into readable text.

Immutability and Defensive Copying

- **Immutable:** state cannot change after creation.
- **Why:** safe to share, thread-safe, fewer bugs.
- **Defensive copy:** copy collections on the way in and out.
- **Recipe:** final class, final fields, no setters.

```
public final class Order {
    private final List<Item> items;

    public Order(List<Item> items) {
        // copy in: caller's later changes
        // cannot affect us
        this.items = List.copyOf(items);
    }

    public List<Item> items() {
        return items;   // already unmodifiable
    }
}
```

Demo: Immutability and Defensive Copying

What we build

- **LeakyOrder:** keeps the caller's own list.
- **The bug:** the cart changes after checkout and the total changes with it.
- **Order:** final class, final fields, and a private copy of the list.

How the concept helps

- **Defensive copy** makes later changes by the caller harmless.
- **Unmodifiable list** blocks edits from outside.
- **Immutable objects** are safe to share and thread-safe.

Records: Compact Data Classes

- **record** auto-creates constructor, getters, equals, hashCode, toString.
- **Immutable:** fields cannot change.
- **Compact constructor** validates input.
- **Great for** value objects like Money and Address.

```
public record Money(BigDecimal amount,
                    String currency) { }

Money m = new Money(new BigDecimal("99.99"), "INR");
m.amount();    // accessor (no "get" prefix)

public record Address(String city, String zip) {
    public Address {            // compact constructor
        if (city == null || city.isBlank()) {
            throw new IllegalArgumentException(
                "city required");
        }
    }
}
```

Sealed Classes: A Closed Family

- **sealed** limits who may implement or extend.
- **permits** lists the allowed types.
- **Compiler knows** every possible case.
- **Perfect for** payment methods, order states, results.

```
public sealed interface PaymentMethod
        permits Card, Upi, Cash { }

public record Card(String number)
        implements PaymentMethod { }

public record Upi(String id)
        implements PaymentMethod { }

public record Cash()
        implements PaymentMethod { }
```

Nobody else can add a new payment method by accident

Pattern Matching with switch

- **switch** checks the type and gives you a typed variable.
- **No casting** and no long if-else chains.
- **No default needed** for a sealed type - compiler checks all cases.
- **Add a new type?** The compiler tells you where to update.

```
String describe(PaymentMethod p) {
    return switch (p) {
        case Card card ->
            "Card: " + card.number();
        case Upi upi ->
            "UPI id: " + upi.id();
        case Cash cash ->
            "Cash on delivery";
    };
}
```

Demo: Records, Sealed Types, Pattern Matching

What we build

- **Records:** Money, Address, Product and Customer in one line each.
- **Sealed PaymentMethod:** only Card, Upi and Cash are allowed.
- **Checkout:** a switch works out fees, then an order of Items (Product and quantity) runs end to end.

How the concept helps

- **Records** give constructor, equals, hashCode and toString free.
- **Sealed types** close the list of payment methods.
- **Pattern switch** needs no default, and the compiler flags every place to update.

Key Takeaways

1. **Four pillars**
   Encapsulation, inheritance, polymorphism, abstraction.

2. **Design choices**
   Prefer interfaces and composition.

3. **Modern Java**
   Records and sealed types model data clearly.

4. **Safe objects**
   Correct equals/hashCode, immutability.



Happy Learning :)